SYSTEM WALKTHROUGH

DocQuorum

From document to structured JSON

The task: extract the fields and repeated records requested by a schema from a PDF or image, while preserving what the document actually prints.

DocQuorum prepares several representations of the document, chooses a suitable reader, repairs each resulting JSON independently, and—when several candidates remain—selects the candidate with the strongest agreement with the others.

1 Define the extraction task

The input is a document D, a schema S (or a JSON template), and optionally a task description T. The intended output is a JSON object Y whose fields, types, and repeated records follow S and whose values are supported by D:

$$(D, S, T) \longrightarrow Y.$$

The schema supplies both structure and instructions. For example, it can request a purchase-order number, a supplier name, and every line item with its quantity and unit price. Descriptions can specify grouping, units, missing values, or whether a page number means the PDF index or the printed folio.

	Example input	Intended output
Document	Purchase order with a header and item table	Values grounded in those pages
Schema	Order number; supplier; list of items	One object with those keys and an item array
Optional task	“Extract every ordered item.”	Task text becomes the schema’s top-level description

What a “reading” means

A reading is one candidate JSON for the task. It may require one model call or many calls over smaller pieces. A parse is different: MinerU produces page text and HTML tables, which a reader still has to interpret under the task schema.

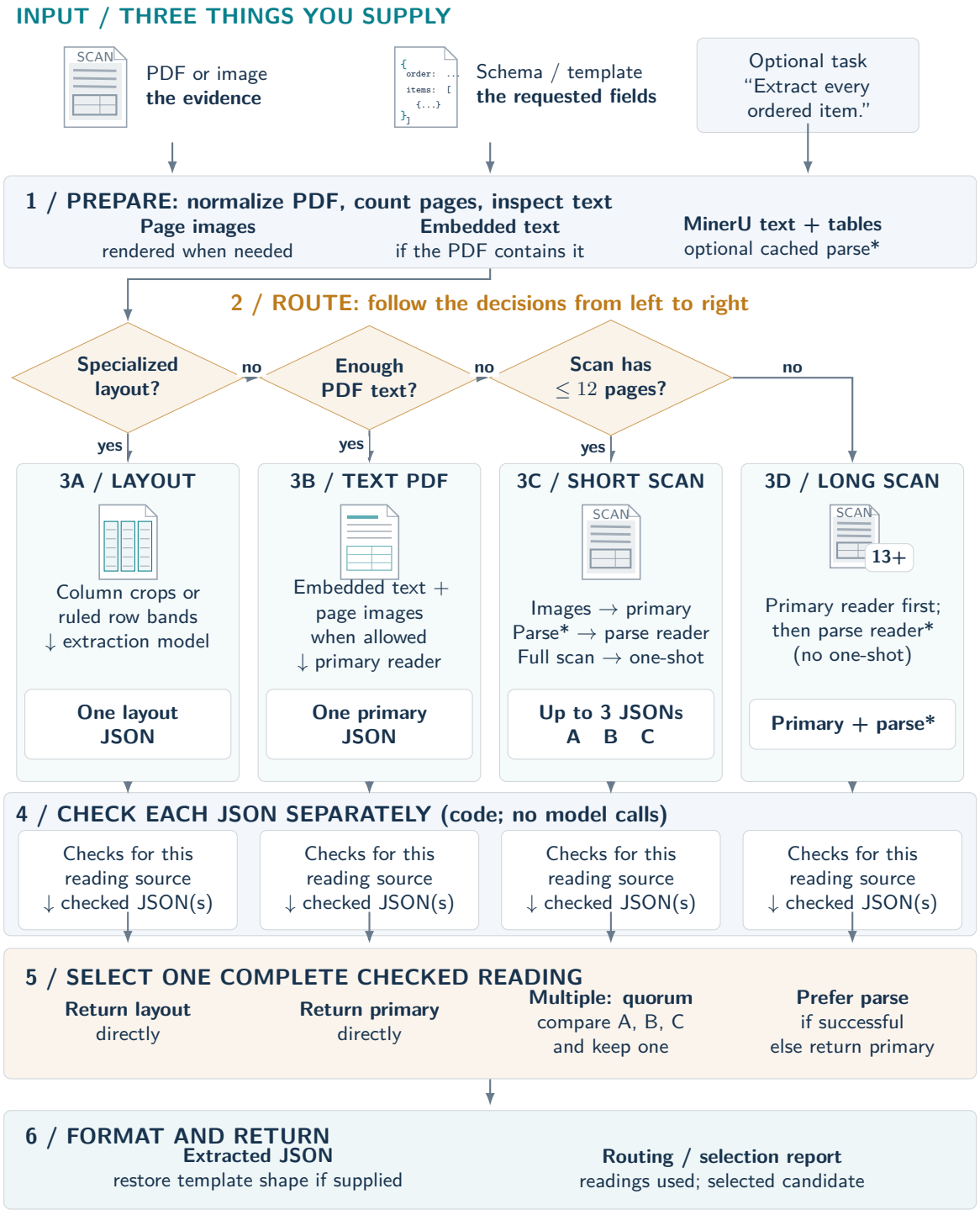
How to read this guide

Start with the illustrated flow on page 2. The following sections explain each operation. Page 8 follows one scanned purchase order through three separate readings; page 9 pictures how other document types take different routes.

Implementation basis: repository source at `b4ea9a1`. The main flow describes `docquorum.extract.extract`. Benchmark differences are explicit on page 10. This guide explains behavior; benchmark results remain in `results/README.md`.

PIPELINE AT A GLANCE

One document, from input to result



Only one route runs. Short scans keep successful candidates: one returns directly, several enter the quorum, none raises an error. Long scans also fail if no reading succeeds.

* Parse reading needs cached pages and adequate cache coverage. A parse is evidence, not task JSON. The detailed reader uses cached parse text only at $\geq 90\%$ page-count coverage.

This is the main API flow. Checks are conditional repairs, not proof of correctness. Quorum chooses a whole candidate; it does not assemble fields from different candidates.

STEPS 1–2 / PREPARE AND ROUTE

Turn the document into usable evidence

Step 1. Normalize and inspect

Input: PDF or image, schema/template, optional task.

Operation: copy a PDF into a temporary workspace, or convert an image to an RGB PDF at 150 dpi. Convert a template to a schema. If supplied, the task replaces the top-level schema description. Count pages and inspect the PDF text.

The routing test is `len(plain_text.strip()) >= 50 * max(1, n_pages)`. It measures aggregate text availability, not OCR accuracy. A scanned PDF with enough embedded OCR text takes the text-present route. Mixed documents are classified as a whole.

For a document below this threshold, the API probes the parser endpoint. If available, MinerU processes the pages and caches each page as `NNNN.txt`. Text blocks remain text; tables retain HTML cell structure. Existing cached files are reused. The document cache key is derived from its bytes.

Output: normalized PDF, page count, text flag, text-layer strings, optional cached parse. Page images are rendered when a reader requests them.

Step 2. Select a reader route

Priority	Condition	Decision
1. Column flow	At least two pages; at least three nearly equal-width columns on at least half of sampled pages	Column reader
2. Ruled scan	No sufficient text layer; cached tables map to schema fields with conversion ratio ≥ 0.9	Ruled reader
3. General	Neither specialized condition holds	General reading route

The column test samples up to four pages and requires column-width coefficient of variation below 0.12. The ruled test's conversion ratio compares converted records with parsed table body rows on pages not marked bad by the converter. It is a routing heuristic, not a measured extraction accuracy.

Keep the representations distinct

Artifact	Contains	Used by
PDF text layer	Characters already embedded in the PDF	Routing, reader, checks
Page image / crop	Rendered visual evidence	Parser and visual readers
MinerU parse	Machine-read text and HTML tables	Parse reader, layout test, some checks
Candidate JSON	Values mapped to the task schema	Checks and quorum

Data boundary: answer keys do not enter preparation, routing, reading, checks, or candidate selection. Benchmark scoring is a separate operation after extraction.

STEP 3 / GENERAL READER

Plan, read, inspect, and reduce the scope

Input: schema and PDF, with either text-layer pages or cached parse pages. The primary reader uses page images when text is insufficient. Parse mode uses cached text only if its page count covers at least 90% of the extracted page-text count; otherwise it falls back to the text layer.

Output: candidate JSON, call statistics, and a plan description. The reader tries the following plans in order, returning when a plan is accepted.

Plan	When attempted	What it does
1. Whole document	Estimated input $\leq$ 100,000 tokens and output $\leq$ 140,000; image-only input has $\leq$ 12 pages	Send full schema and available evidence in one call; inspect completion and missing keys
2. Schema groups	Whole plan did not return; compact schema exceeds 60,000 characters; text exists or $n \leq 12$	Split top-level fields with their definitions; read groups, using page windows when needed
3. Text windows	Earlier plans did not return; text exists and schema has no top-level arrays	Read consecutive windows and merge filled values
4. Scalars + lists	Earlier plans did not return	Read document-level fields; then read each top-level array in page chunks

What each model call sees

Whole-document calls include images for up to 12 pages. A parse-based scan of 13–24 pages can use paired-page images. Longer whole-document calls use text only. Partitioned long scans can use windows of up to eight page images; ordinary text windows target 60,000 tokens. OCR text is labeled as machine-read when accompanied by images.

In the chunk plan, text chunks contain text only; image-mode chunks contain up to two page images. Chunk size is one or two pages, estimated from row density and field count against a 9,000-token output target. Fields whose names contain "page" force one-page chunks. Results are collected in page order; earlier chunk predictions are not fed into later calls.

What causes a retry or fallback

- A whole reply ending with `finish_reason=length` may be retried at 32,000 output tokens. Continued truncation triggers the next plan.
- A guided whole reply missing schema keys is retried with all keys required; a usable retry replaces it.
- Truncated *text* chunks split recursively, up to depth four; keep the halves' result only if it has more rows. Image chunks are not recursively split by this logic.
- A text chunk with at least five estimated rows but no extracted records gets one extra attempt with a row-count reminder.

Schema/window merging fills empty values, keeps the first nonempty scalar, and keeps the longer list. It does not concatenate window lists. In the chunk plan, scalar-only evidence is limited to the first two pages in image mode, or first three and last two pages if text exceeds 240,000 characters.

STEP 3 / ALTERNATIVE READERS

Read the geometry that defines a record

Column reader: preserve the reading order

Input: page images from a document routed as column flow, and the schema's main array.

Operation:

1. Render each page at 300 dpi and crop its detected columns.
2. Read each crop from top to bottom with the item schema. The decoding schema requires the item fields and permits a null lead field.
3. Collect records in page order, then column order. Assign the physical PDF page index to `source_page`.
4. If a record has an empty lead field and a preceding record exists, treat it as a continuation: fill empty fields in the previous record and append string content.

Output: one candidate object containing the main array. The lead field is the first required string field whose name does not contain "page", with a fallback to the first property. A mistaken empty lead field can therefore cause an incorrect join.

Ruled reader: use printed lines to count records

Input: a routed scanned table and the schema's main array.

Operation:

1. Render the page at 200 dpi, deskew it, and detect horizontal rules.
2. Identify a header and row bands of up to 12 rows.
3. Stack the header above each band so every call sees the column labels.
4. Decode an array with `minItems = maxItems = n`, where n is the detected row count for that band.
5. Attach the physical page index and concatenate records in page order.

Output: one main-array candidate, then the layout check chain. A page with no detected row bands yields no records from this reader. The row constraint relies on correct rule detection; it does not establish that the values are correct.

One-shot reader: an additional candidate for short scans

The general API attempts the ExtractBench `VLLMExtractProvider` reader for documents without sufficient text and with at most 12 pages. It uses that provider's prompt and rendering configuration (150 dpi) with the configured extraction model. Its JSON receives the one-shot check chain.

This is a separate reading strategy. It does not revise the primary reader's JSON. If this call raises an exception, the API logs the failure and continues with available candidates.

Why these readers are separate

A column crop makes small print readable and preserves column order. A ruled band provides a physical count of rows. The one-shot strategy provides another interpretation of the same short scan. Specialized layout output is returned directly after checks; it does not compete in the general quorum.

STEP 4 / CHECK EACH CANDIDATE

Repair JSON using the schema and evidence

Input: one candidate JSON and a `checks.Doc` object containing the schema and lazily loaded text layer, reading-order text, and cached parse.

Operation: run deterministic transformations in a fixed order. Every transformation receives a fresh JSON copy. There are no model calls or answer keys.

Output: one checked candidate, still separate from every other reading.

Candidate source	Exact chain
Primary	Repairs → Output checks
Parse	Spacing → Repairs → Output checks
One-shot / layout	Output checks only

Spacing and repairs, in order

Module	Intended transformation; applies only when its guards match
`spacing`	Restore text-layer spacing when a parsed string differs only in spaces.
`carry_down`	Propagate a group value to later rows when the schema describes carry-down behavior.
`page_index`	Normalize `source_page`; translate physical indices to printed folios where requested.
`dedupe`	Deduplicate on a key identified by the list's uniqueness instructions.
`stacked_rows`	Join records split across multiple printed lines as described by the schema.

Output checks, in order

Module	Intended transformation; applies only when its guards match
`conform`	Fill nullable properties; repair missing containers and null/empty-list conventions.
`blank_boxes`	Set a blank checkbox to false when the field description requires it.
`template_rows`	Remove empty preprinted rows or template records.
`copied_rows`	Remove records spuriously copied into another array.
`record_counts`	Cap duplicate records using repetition found in document text.
`printed_pages`	Locate rows through distinctive words and correct printed page labels.
`printed_quotes`	Restore uniquely supported quote/apostrophe forms from the text layer.
`printed_numbers`	Recover uniquely supported printed precision across unit scales; integer-only fields stay unchanged.

These are conditional heuristics. Some cannot act without suitable text evidence. `conform` is not a full JSON Schema validator, and a checked object is not a certificate of factual correctness. For example, missing required scalar values are not universally repaired.

STEP 5 / CHOOSE A READING

Agreement selects one complete candidate

Input: checked candidates for the same document and schema.

Policy: a layout route returns its checked result; a scan longer than 12 pages returns its checked parse reading when available. Otherwise, one candidate is returned directly, several invoke the quorum, and no candidates raises an error.

The quorum calculation

For candidate c, treat each other candidate o as a temporary reference. The ExtractBench scorer computes value precision $P(c \mid o)$ and recall $R(c \mid o)$ under the schema. The selector explicitly supplies empty field rules and repeated-structure rules; it does not use the benchmark's annotated answer or custom matching rules.

$$F_2(c \mid o) = \frac{5P(c \mid o)R(c \mid o)}{4P(c \mid o) + R(c \mid o)}, \qquad A(c) = \frac{1}{m-1}\sum_{o \neq c} F_2(c \mid o).$$

If precision or recall is nonpositive, the pair contributes zero. The candidate with maximum $A(c)$ wins. Scores are rounded to nine decimals before comparison; ties go to insertion order (primary, parse, one-shot in the API).

Recall is weighted more heavily: a candidate is rewarded for retaining values found by other readers. Pairwise comparisons are directional. The selected JSON remains a complete checked candidate; fields are not voted on or stitched together.

Worked selection example

Consider a simplified array where every item is a distinct matching value. Candidate A contains `[a,b]`, B contains `[a,b,c]`, and C contains `[a,b,c]`. This illustrates the arithmetic; real structured matching uses the scorer.

Candidate	Against first peer	Against second peer	Mean F_2
A	B: $P = 1, R = 2/3$	C: $P = 1, R = 2/3$	0.714286
B	A: $P = 2/3, R = 1$	C: $P = 1, R = 1$	0.954545
C	A: $P = 2/3, R = 1$	B: $P = 1, R = 1$	0.954545

B wins the tie with C because B is earlier. Its complete array is returned. Agreement cannot reveal an error shared by all readings; a high agreement score is not a calibrated probability of correctness.

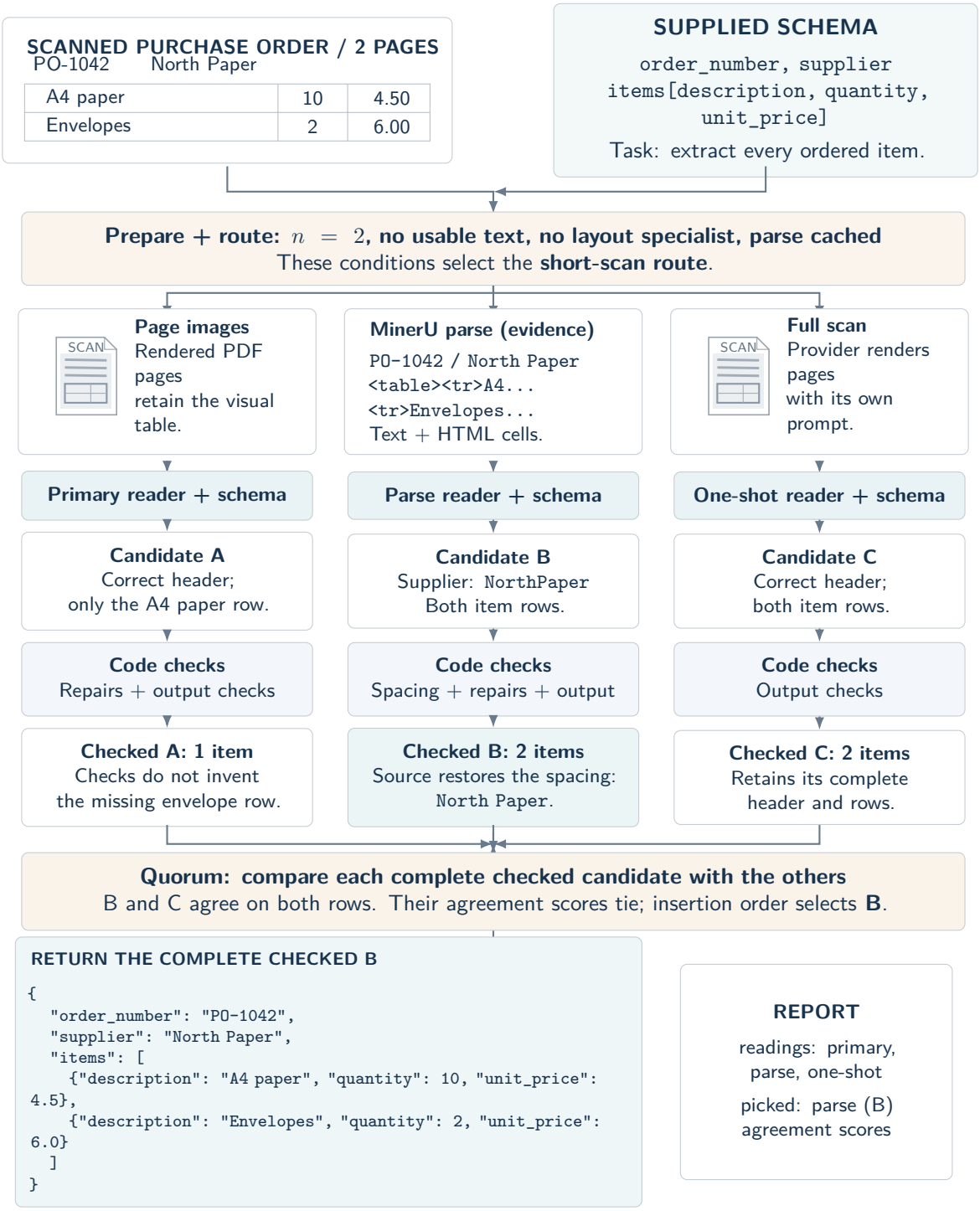
Step 6. Format and return

For a JSON Schema input, return the selected object. For a template input, `fill_template` restores the original template shape and uses empty strings for absent leaves. The Python API returns `(result, report)`. A multi-candidate report includes candidate names, the pick, and rounded agreement scores. Direct returns include a readings list; the long-scan direct return reports only "parse", even though primary was attempted first.

END-TO-END EXAMPLE

Follow the evidence into the final JSON

This is a synthetic walkthrough, not a benchmark result. Assume there is no usable text layer, no specialized layout route, and the parser is available.



Illustrative values and candidate errors. The spacing repair needs matching source evidence. Side-by-side branches show independent interpretations, not an assertion of concurrent execution. Quorum does not combine A's header with B's rows.

ROUTING COMPARISONS

Four documents, four different paths

Follow one pictured case at a time. The layout test comes first; text availability and page count decide the remaining routes.

DIGITAL PDF OR SCAN
ENOUGH EMBEDDED TEXT



No specialist selected.
Enough embedded text selects the text route, even if the PDF looks scanned.

Primary reader → Primary checks → Return its JSON

Page images may accompany text. No general quorum is needed for this single reading.

SHORT SCAN
PARSER UNAVAILABLE



No sufficient text; $n \leq 12$.
No parse candidate, but primary and one-shot may still succeed.

Primary + one-shot → Check each separately → Quorum if both succeed

One successful reading returns directly. No successful reading raises an error.

LONG SCAN
SUCCESSFUL CACHED PARSE



No sufficient text; e.g. $n = 20$.
Try primary, then the parse reading.
A successful parse takes priority.

Read primary, then parse → Check each separately → Return checked parse

No one-shot or quorum. Primary is the fallback if the parse reading is unavailable.

SPECIALIZED LAYOUT
SELECTED FIRST



Equal-width columns or a routed ruled scan. This decision has priority over the general text/scan routes.

Column / ruled reader → Layout checks → Return its JSON

One geometry-aware candidate. The API does not compare it with general-reader candidates.

The parse contains text and HTML tables; a model must still map that evidence into the task schema. Each candidate is checked independently before selection. A high agreement score cannot detect an error shared by every candidate.

EXECUTION AND SCOPE

Connect the guide to the code

Run a document through the API

From the repository root, with dependencies and model endpoints configured as described in the repository README:

```
python -m docquorum.extract invoice.pdf \
  --schema invoice_schema.json -o invoice.json
```

```
from docquorum.extract import extract
result, report = extract("invoice.pdf", schema_dict)
```

The CLI writes the JSON to the chosen file (or stdout) and the final report to stderr. Some reader diagnostics use direct prints, so `-o` is the reliable way to capture a separate JSON artifact.

Implementation map

Source under `docquorum/`	Responsibility
`extract.py`	Preparation, route, candidate lifecycle, final formatting
`tools/pdf.py`, `tools/parser.py`	Evidence extraction, rendering, cached page parses
`agents/layout.py`	Specialized layout decision
`agents/reader.py`, `agents/whole.py`	General reader plans, retries, chunks, windows
`agents/column_reader.py`, `agents/ruled_reader.py`	Specialized visual readings
`agents/oneshot.py`	Additional short-scan provider
`checks/__init__.py`, `checks/*.py`	Exact ordered chains and transformations
`quorum.py`	Pairwise agreement and whole-candidate selection
`tools/schema.py`	Schema decomposition and template formatting

Benchmark runners have different orchestration

ExtractBench: `benchmarks/extractbench/run.sh` runs primary over the split first, then parse for scans, then one-shot/quorum for short scans. Layout results overwrite general results on medium/long splits only. The script reads cached parses; prepare them with `python -m docquorum.tools.parser SPLIT`. Unlike the API, it does not create missing parses automatically. Its quorum command may also copy parse-only long-scan results. Scoring uses answer keys only after extraction.

UNIKIE: `benchmarks/unikie/unikie_eval.py` uses a separate template-based image protocol and six readings: image, one-shot plus parse text, one-shot, parse, and two extra-model one-shots. It uses normalized flattened-value matching for its F_2 quorum. It does not execute the API's layout/reader/check pipeline.

Practical limits

Routing thresholds, row counts, and text repairs are heuristics. Parser caches can contain empty pages. Some call failures can leave partial results; the API excludes general-reader results with recorded server errors, but this does not detect every incomplete output. Specialist failures are not caught by a general-reader fallback. Inspect important extractions against the original document; neither structured decoding nor agreement proves completeness.